UNIVERSITY PROJECT · 2026-2

ResumeLens

Formal Language-Based Resume Screening

Computing and Discrete Structures III · Universidad Icesi · 2026-2

PROJECT TEAM

Our Team

01

Gloribeth Renteria

02

Valentina Tangarife

03

Dayanna Fernández

CONTEXT

The Problem

Resume screening is inconsistent and subjective.

- Same skills, different words — JavaScript vs JS vs ECMAScript
- Manual review is error-prone and inconsistent
- Keyword matching is narrow and inflexible
- No standardization across hiring teams

THE GOAL

Automated, formal, and consistent resume screening.

METHODOLOGY

Our Solution

Six formal computational stages — one formal model, one job.

01 **REGEX** — 02 **FST** — 03 **DFA** — 04 **TIER** — 05 **FA** — 06 **CFG**

One formal model, one job. Profiles are data, not code.

STAGE 01 · GLORIBETH

Stage 1: Extraction

Regular Expressions — parse resume text to extract contact, skills, experience, and education.

INPUT → OUTPUT

Raw resume text

```
["REST APIs", "JS",
 "PostgreSQL", "Git",
 "React"]
```

PRINCIPLE

Extraction does NOT decide equivalence.

Regex captures surface forms. Deciding whether "JS" and "JavaScript" are the same is the job of Stage 2A.

Stage 2A: Normalization

Finite-State Transducers canonicalize every spelling variant.

$$M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$$

120+ transducers maintained as data.

INPUT VARIANTS

JS

Java Script

ECMAScript

CANONICAL OUTPUT

JAVASCRIPT

Stage 2B: Classification

Profile matching with weighted scoring.

```
Profile_Score  =  (0.7 × Category_Coverage)  +  (0.3 × Skill_Specificity)
```



Coverage 70% **Specificity 30%**

PROFILE 01

Full Stack

PROFILE 02

ML Engineer

PROFILE 03

Backend

PROFILE 04

Data Engineer

STAGE 02C · VALENTINA

Stage 2C: Ranking & Tiering

Candidate ranking system across four decision tiers.

TIER	THRESHOLD	MATCH QUALITY	ACTION
TIER_1	≥ 0.85	Excellent match	Interview immediately
TIER_2	≥ 0.70	Good match	Strong candidate
TIER_3	≥ 0.55	Partial match	Consider if needed
REJECTED	< 0.55	Poor match	Not suitable

Stage 3: Recognition

Finite Automata validation — each profile is a regular language.

Full Stack	`LANGUAGE+ FRONTEND+ BACKEND+ DATABASE+ DEVOPS*`
ML Engineer	`LANGUAGE+ ML_FRAMEWORK+ DATA_LIBRARY+ PROCESSING+`
Backend	`LANGUAGE+ API+ DATABASE+ BACKEND_FRAMEWORK+`
Data	`LANGUAGE+ PROCESSING+ DATABASE+ PIPELINE_TOOL+`

Stage 4: DSL & Output

Context-Free Grammar defines the candidate record; renderers emit multiple formats.

DSL GRAMMAR (textX)

Candidate structure

```
Candidate:
  'name' name=STRING
  'skills' '{' skills+=Skill '}'
  'profile' profile=Profile ;

Skill: name=ID ';' ;

Profile: 'FULL_STACK' | 'ML' |
         'BACKEND' | 'DATA' ;
```

OUTPUT FORMATS

Emit four portable artifacts

- HTML — recruiter-facing report
- Markdown — portable summary
- JSON — machine-readable record
- .rlp — ResumeLens Package

ENGINEERING

Testing & Code Quality

TEST COVERAGE

250+ tests

100% passing

Stage 1	`4 tests`
Stage 2A	`45 tests`
Stage 2B	`38 tests`
Stage 2C	`29 tests`
Stages 3–4	`100+ tests`

PRODUCTION CODE

1,642

lines of Python

Modular architecture · Type hints · Thoroughly documented

WALKTHROUGH

Worked Example

Wednesday Addams — target profile: Full Stack Developer.

STAGE	OUTPUT
Stage 1 · Extraction	`[ JS, React, NodeJS, Postgres, Git ]`
Stage 2A · Normalize	`[ JAVASCRIPT, REACT, NODE_JS, POSTGRESQL, GIT ]`
Stage 2B · Classify	`Full Stack → 0.92`
Stage 2C · Rank	`TIER_1 · Strong match`
Stage 3 · Validate	`ACCEPTED`
Stage 4 · Output	`HTML + MD + JSON + .rlp`

REFLECTION

Key Insights

DESIGN DECISIONS

What worked

- One model per stage
- Profiles as data
- Type safety end-to-end
- Clear handoffs between stages
- Extensible design

LIMITATIONS & FUTURE

What's next

- Regex depends on input quality
- FST variants require manual updates
- DFA models are static
- Future: ML-based synonyms
- Future: confidence scores

WHY IT MATTERS

Real-World Impact

For Recruiters

- Automatic screening
- Consistent criteria
- Less bias

For Candidates

- Fair evaluation
- Skills recognized
- No gatekeeping

For Companies

- Scalable
- Auditable
- Portable

Portable to HR systems, ATS platforms, and job boards.

FINAL NOTES

Conclusion

PHILOSOPHY

Clear Jobs. Honest Limits.

Each formal model has one clear job — and acknowledges what it cannot decide.

KEY TAKEAWAY

Formal languages bring

RIGOR, CLARITY, and EXTENSIBILITY.